

Konzeptionspapier: Projektideen & Erweiterungen

Detaillierte Aufschlüsselung potenzieller Features für das Feedback- und Verwaltungs-Tool zur Stärkung der Digitalisierung im Schulalltag.

1. E-Mail- & Benachrichtigungssystem (SMTP)

Ziel: Automatisierte Informationsflüsse zwischen Kollegium, Sekretariat und Schulleitung, ohne dass das System permanent manuell geprüft werden muss.

Umsetzung & Architektur:

- **Bibliothek:** Installation des PHPMailer -Pakets via Composer (`composer require phpmailer/phpmailer`). Das standard `mail()` von PHP ist fehleranfällig und landet oft ungefragt im Spam.
- **Datenbank:** Die Tabelle `app_settings` wird um SMTP-Zugangsdaten (Host, Port, User, Passwort) erweitert.
- **Trigger:** In Dateien wie `admin_action.php` oder `krankmeldung.php` wird eine Hilfsfunktion `send_notification_email($to, $subject, $body)` eingebaut, die direkt nach dem Schreiben der Daten in die DB aufgerufen wird.
- **Cronjob:** Für die heute entworfene automatische Liste (Krankmeldungen & Ausflüge) wird eine Datei `cron_report.php` erstellt. Diese wird vom Betriebssystem des Servers z.B. täglich um 07:00 Uhr über Crontab ausgeführt, liest die DB aus und mailt das fertige PDF an das Sekretariat.

2. Kalender-Integration (iCal/ICS Export)

Ziel: Alle relevanten Abwesenheiten (Krankheit, Freistellung, Ausflüge) direkt in den Schulkalender (Outlook, Nextcloud, Apple) der Schulleitung pushen.

Umsetzung & Architektur:

- **Endpunkt:** Erstellung einer `calendar_feed.php`. Diese Seite erfordert kein klassisches Login, sondern nutzt einen zufällig generierten Secret-Token in der URL (z.B. `?token=A1b2...`), um den Kalender vor Unbefugten zu schützen.
- **Datensammlung:** Die Datei führt SELECT-Abfragen über alle *genehmigten* Ausflüge und aktuellen Krankmeldungen aus.
- **Formatierung:** PHP schreibt keine HTML-Antwort, sondern gibt den HTTP-Header `Content-type: text/calendar` aus und erzeugt eine Textstruktur im standardisierten vCard/ICS-Format (mit `BEGIN:VCALENDAR`, `DTSTART`, `DTEND`, `SUMMARY`).

- **Nutzen:** Das Sekretariat abonniert diesen Link einmalig. Jede neue Krankmeldung taucht 5 Minuten später auto-synchronisiert im Outlook der Schule auf.

3. Single Sign-On (SSO) & Schul-Login-Anbindung

Ziel: Lehrkräfte müssen sich kein neues Passwort merken. Sie loggen sich mit ihrem bestehenden Schul-Account (IServ, Microsoft 365, Mebis, Nextcloud) ein.

Umsetzung & Architektur:

- **Analyse:** Zuerst muss geklärt werden, welchen Identity Provider die Schule nutzt. M365 nutzt OpenID Connect (OIDC), IServ bietet oft SAML oder LDAP an.
- **Bibliothek:** Für OIDC nutzt man `jumbojett/openid-connect-php`, für SAML nutzt man `onelogin/php-saml`.
- **Workflow:** Die `login.php` wird umgebaut. Statt Formularfeldern gibt es einen Button "Mit Schul-Account einloggen". Dieser leitet Nutzer auf die Microsoft/IServ-Anmeldeseite um. Nach erfolgreichem Login werden sie mit einem Token sicher validiert und zurückgeschickt.
- **Datenbank-Sync:** Das System liest E-Mail und Namen aus dem Token. Existiert der Lehrer (`teachers`) anhand der E-Mail noch nicht, wird er automatisch angelegt.

4. Visuelle Statistiken & Diagramme (Chart.js)

Ziel: Krankheitswellen, Verteilung von Ausflügen auf Wochentage oder das Schülerfeedback visuell aufbereiten, statt nur nackte Tabellen zu zeigen.

Umsetzung & Architektur:

- **Bibliothek:** Einbetten der extrem performanten JavaScript-Bibliothek `chart.js` über ein CDN im Twig-Template.
- **Backend (Datenbereitstellung):** Erstellen einer sauberen JSON-API `api_statistics.php`, die aggregierte Daten ausliefert. Beispiel: `SELECT MONTH(date_from), COUNT(*) FROM sick_leave_reports`.
- **Frontend:** Im Dashboard wird HTML `<canvas>` angelegt. Mit JavaScript holt sich die Seite die JSON-Daten asynchron (Fetch-API) und rendert wunderschöne, interaktive Balken- oder Tortendiagramme.

5. Progressive Web App (PWA) für Smartphones

Ziel: Das Werkzeug soll sich wie eine echte Smartphone-App verhalten inklusive Icon, sodass diese jederzeit als Begleiter parat ist.

Umsetzung & Architektur:

- **Web App Manifest:** Eine `manifest.json` im Root-Verzeichnis anlegen. Darin stehen Name, Start-URL, primäre Farben und Pfade zu den App-Icons (PNGs in verschiedenen Größen).
- **Service Worker:** Eine `sw.js`, die Basis-Assets (CSS/Bilder) offline im Local-Cache des Handys abspeichert, sodass die App sofort und blitzschnell lädt, selbst bei schlechtem Internet in der Schule.
- **UI Anpassungen:** Die bestehenden Twig-Ansichten noch weiter "Mobile First" trimmen. Elemente und Buttons groß genug für Touchscreens skalieren.

6. Vertretungsmaterial (Cloud-Anbindung)

Ziel: Bei einer Krankmeldung sofortiges Bereitstellen von Aufgaben für die vertretende Lehrkraft, idealerweise DSGVO-konform über die Schul-Cloud.

Umsetzung & Architektur:

- **Einfacher Weg (Link-Feld):** In das Krankmeldungsformular wird ein Feld "Link zur Nextcloud/Material" hinzugefügt. Die verhinderte Lehrkraft kopiert den Link zu einem ihrer Cloud-Ordner hinein. Dieser taucht sofort im Vertretungs-Dashboard auf.
- **Automatisierung WebDAV (API):** Sobald jemand auf "Meldung abgeben" drückt, spricht das System im Hintergrund automatisch die Schul-Nextcloud an (via WebDAV-Request), legt strukturiert einen Ordner "Vertretung_Mustermann_Datum" an und schickt Links mit den passenden Schreib- und Berechtigungen an die Beteiligten.